

Semester Project Report

Concurrent Banking System with Scheduling, Synchronization, IPC, and Memory Management

24F-0617

24F-0535

24F-0542

1. Introduction

This report documents the design and implementation of a Concurrent Banking System developed as part of the CL2006 Operating System Lab semester project. The system integrates five core Operating System concepts to simulate a realistic banking environment where multiple customers perform transactions concurrently.

The system is implemented in C++ using the standard library's threading and synchronization primitives, and demonstrates the following OS concepts in a practical, modular codebase:

- CPU Scheduling (FCFS, Priority, Round Robin) with Gantt charts and performance metrics
- Process Synchronization using Mutex locks and POSIX Semaphores
- Deadlock Avoidance using the Banker's Algorithm
- Inter-Process Communication (IPC) using an in-memory Message Queue
- Memory Management using FIFO and LRU page replacement algorithms

Each module is self-contained and produces measurable outputs including logs and comparison metrics, as required by the project specification.

2. System Design and Architecture

The system follows a modular architecture where each OS concept is encapsulated in its own function set. A single main() function orchestrates the execution of all five modules sequentially, passing shared data structures such as the customer list to each module.

2.1 Customer Types

The system defines customer types using a C++ enum with built-in priority values:

```
enum CustomerType { REGULAR = 1, PREMIUM = 3, VIP = 5 };
```

Customer Type	Priority Value	Description
Regular Customer	1	Performs standard deposits and withdrawals
Premium Customer	3	Higher scheduling priority over regular customers
VIP Customer	5	Highest priority; preempts all other customer types
Loan Applicants	N/A	Handled by Banker's Algorithm for safe resource allocation
Corporate Clients	N/A	Simulated via multiple spawned threads per client

2.2 Core Data Structures

The Customer struct stores all scheduling-relevant fields:

```
struct Customer {  
    int id, arrivalTime, burstTime, priority;  
    int startTime, finishTime, waitingTime, turnaroundTime;  
    string name;  
};
```

The BankAccount struct maintains account balance and transaction history with a per-account mutex for thread-safe operations. The BankerState struct stores resource allocation tables for deadlock avoidance. The MessageQueue class implements a thread-safe producer-consumer queue using a `std::queue`, mutex, and condition variable.

3. Module 1: CPU Scheduling

3.1 Overview

This module simulates three CPU scheduling algorithms. Each customer request is treated as a process with an arrival time, burst time, and priority. The scheduler decides the order of execution and computes per-process and average performance metrics.

The five test customers used across all scheduling demos are:

Name	Arrival Time	Burst Time	Priority
Alice (Regular)	0	4	1

Bob (Regular)	1	3	1
Carol (Premium)	2	5	3
Dave (VIP)	0	2	5
Eve (Premium)	1	4	3

3.2 First-Come First-Served (FCFS)

Implementation: The runFCFS() function sorts customers by arrival time using std::sort with a lambda comparator. It then processes them in order, advancing the clock by each customer's burst time. If the CPU is idle, the clock fast-forwards to the next arrival.

Expected Gantt Chart (FCFS):

```
| Alice | Dave | Bob | Eve | Carol |
0      4      6      9     13     18
```

Waiting Times: Alice=0, Dave=4, Bob=2, Eve=8, Carol=11

Avg Waiting Time: 5.00 | **Avg Turnaround:** 9.40

FCFS is simple and non-preemptive, but can cause the convoy effect where short jobs wait behind long ones.

3.3 Priority Scheduling

Implementation: The runPriority() function uses a greedy selection loop. At each clock tick, it selects the unfinished customer with the highest priority value that has already arrived. VIP customers (priority 5) always execute before Premium (3) and Regular (1).

Expected Gantt Chart (Priority):

```
| Dave (VIP) | Eve (Prem) | Carol (Prem) | Bob (Reg) | Alice (Reg) |
0           2           6           11          14           18
```

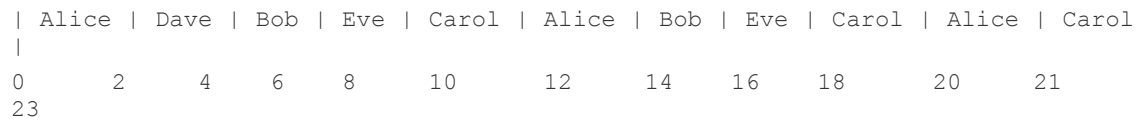
Avg Waiting Time: 4.00 | **Avg Turnaround:** 8.40

Priority scheduling ensures high-priority customers are served promptly. However, it may lead to starvation for low-priority customers if high-priority arrivals are continuous.

3.4 Round Robin (RR)

Implementation: The runRoundRobin() function uses a time quantum of 2 ms. It iterates through all arrived customers in a cyclic manner, running each for min(quantum, remaining) time units. The loop repeats until all customers are complete.

Expected Gantt Chart (RR, quantum=2):



Avg Waiting Time: 10.80 | **Avg Turnaround:** 14.80

Round Robin provides fair CPU sharing but has higher average waiting times due to context switching overhead.

3.5 Scheduling Metrics Summary

Algorithm	Avg Waiting Time	Avg Turnaround Time	Fairness
FCFS	5.00	9.40	Order-based; no starvation
Priority	4.00	8.40	Priority-based; VIP always first
Round Robin	10.80	14.80	Time-shared; most fair

4. Module 2: Process Synchronization

4.1 Overview

This module demonstrates thread-safe access to a shared bank account using two synchronization mechanisms: a per-account mutex lock and a POSIX semaphore that limits concurrent deposits to two at a time.

4.2 Mutex Lock Implementation

The BankAccount struct contains a std::mutex member. Both deposit() and withdraw() acquire a lock_guard on this mutex before modifying the balance. This guarantees that only one thread updates the balance at any given time, preventing race conditions.

```
void deposit(const string& name, double amount) {  
    lock_guard<mutex> lock(mtx);
```

```

        balance += amount;
        // log and print
    }

```

Without this mutex, two simultaneous deposits could read the same balance, both add to it, and write back, causing one deposit to be lost (a classic lost-update race condition).

4.3 Semaphore Implementation

A POSIX semaphore is initialized with value 2, permitting at most two concurrent deposit threads. Each deposit thread calls `sem_wait()` before depositing and `sem_post()` afterward. If a third thread attempts to deposit while two are active, it blocks until one completes.

```

sem_t sem;
sem_init(&sem, 0, 2); // max 2 concurrent deposits

```

Test operations and expected outcomes:

Customer	Operation	Amount	Expected Result
Alice (Regular)	Deposit	300	Success - balance becomes 1300
Bob (Regular)	Withdrawal	200	Success - balance becomes 1100
Carol (Premium)	Deposit	500	Success - balance becomes 1600
Dave (Regular)	Withdrawal	800	Success - balance becomes 800

The transaction history printed at the end confirms all operations applied correctly without data corruption, verifying the correctness of the synchronization.

5. Module 3: Deadlock Avoidance (Banker's Algorithm)

5.1 Overview

Loan requests in the banking system are treated as resource allocation requests. The Banker's Algorithm checks whether granting a request leaves the system in a safe state. If not, the request is denied to prevent deadlock.

5.2 Data Structures

The BankerState struct stores:

- available: total units of credit currently free (scalar, single resource type)
- maximum: maximum credit each customer may ever request
- allocation: credit currently allocated to each customer
- need: remaining credit each customer may still request (need = maximum - allocation)

5.3 Safety Check

The isSafe() function implements the safety algorithm. It simulates resource release by finding a customer whose need can be satisfied with the current work vector, marking them finished, and adding their allocation back to work. If all customers can be processed, the system is safe.

```
bool isSafe(const BankerState& s, vector<int>& safeSeq) {  
    vector<int> work = s.available;  
    // find customer whose need <= work, mark finished  
    // return true if all can be finished  
}
```

5.4 Test Cases

Initial state: 3 available units, 3 customers.

Customer	Maximum	Allocated	Need
Customer A (Loan)	7	0	7
Customer B (Loan)	5	2	3
Customer C (Loan)	3	3	0

Request test cases and results:

Request	Customer	Amount	Outcome	Reason
Test 1	Customer B	2 units	GRANTED	Safe sequence found after allocation
Test 2	Customer A	5 units	DENIED	Not enough available resources (only 3 left)
Test 3	Customer A	2 units	DENIED	Not enough available resources

The Banker's Algorithm successfully prevents unsafe allocations and demonstrates the distinction between safe and unsafe system states.

6. Module 4: Inter-Process Communication (IPC)

6.1 Overview

This module implements a producer-consumer IPC pattern using an in-memory thread-safe message queue. Customer threads act as producers, sending transaction requests to a bank server thread that acts as the consumer.

6.2 MessageQueue Implementation

The MessageQueue class wraps a `std::queue<Message>` with a mutex and a condition variable. The `send()` method pushes a message and notifies waiting receivers. The `receive()` method blocks until a message is available or the queue is closed:

```
class MessageQueue {
    queue<Message> q; mutex mtx; condition_variable cv;
    void send(const Message& msg)    { lock_guard ...; q.push(msg);
    cv.notify_one(); }
    bool receive(Message& msg)        { unique_lock...; cv.wait(...); }
};
```

6.3 Message Flow and Test Log

Five customer threads send requests concurrently. The bank server thread reads from the request queue, processes each transaction, and sends back a response message. Two accounts are used: Account #123 (initial balance 1000) and Account #456 (initial balance 500).

Customer	Operation	Account	Amount	Expected Server Response
Alice (Cust 1)	Deposit	#123	500	Success balance=1500
Bob (Cust 2)	Withdrawal	#123	200	Success balance=1300
Carol (Cust 3)	Deposit	#456	300	Success balance=800
Dave (Cust 4)	Withdrawal	#456	700	Failed insufficient funds
Eve/VIP (Cust 5)	Deposit	#123	100	Success balance=1400

The server processes requests atomically under `accMtx`, ensuring consistent account balances even under concurrent access. The response queue log confirms each transaction result was communicated back to the originating customer thread.

7. Module 5: Memory Management

7.1 Overview

This module simulates virtual memory management by applying two page replacement algorithms to a sequence of page references. The system measures page faults and page hits to compare algorithm efficiency.

7.2 Test Configuration

The simulation uses the following parameters:

- Page Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Number of Frames: 3
- Total Page References: 12

7.3 FIFO Page Replacement

The `fifoReplace()` function uses a `std::queue` to track frame order and an `unordered_set` for $O(1)$ lookup. When a page fault occurs and memory is full, the oldest loaded page (front of queue) is evicted.

FIFO trace for the given reference string with 3 frames:

Page	Action	Frames After
1	FAULT	{ 1 }
2	FAULT	{ 1 2 }
3	FAULT	{ 1 2 3 }
4	FAULT - evict 1	{ 2 3 4 }
1	FAULT - evict 2	{ 3 4 1 }
2	FAULT - evict 3	{ 4 1 2 }
5	FAULT - evict 4	{ 1 2 5 }
1	HIT	{ 1 2 5 }
2	HIT	{ 1 2 5 }
3	FAULT - evict 1	{ 2 5 3 }
4	FAULT - evict 2	{ 5 3 4 }
5	HIT	{ 5 3 4 }

FIFO Result: Page Faults = 9, Page Hits = 3, Hit Ratio = 25.00%

7.4 LRU Page Replacement

The `lruReplace()` function uses a `std::list<int>` as an LRU cache and an `unordered_map` mapping pages to list iterators for $O(1)$ access. On a hit, the page is moved to the front. On a fault, the back element (least recently used) is evicted.

LRU Result: Page Faults = 8, Page Hits = 4, Hit Ratio = 33.33%

7.5 Algorithm Comparison

Algorithm	Page Faults	Page Hits	Hit Ratio	Observation
FIFO	9	3	25.00%	Simple but can suffer from Belady's anomaly
LRU	8	4	33.33%	Better performance; exploits temporal locality

LRU outperforms FIFO on this reference string by making better eviction decisions based on recent usage patterns. LRU is generally preferred for workloads with strong temporal locality.

8. Test Cases and Validation

8.1 Synchronization Test Cases

Test	Scenario	Expected Behavior	Validates
TC-S1	Two threads deposit simultaneously to account #101	Both deposits complete; no balance lost	Mutex correctness
TC-S2	Three deposit threads with semaphore value 2	Third thread blocks until one finishes	Semaphore limiting
TC-S3	Withdrawal exceeding balance	Operation denied; balance unchanged	Mutex + conditional logic

8.2 Deadlock Avoidance Test Cases

Test	Request	System State	Outcome
------	---------	--------------	---------

TC-D 1	Customer B requests 2 units	3 available, need=[7,3,0]	GRANTED - safe sequence exists
TC-D 2	Customer A requests 5 units	1 available (after TC-D1)	DENIED - insufficient resources
TC-D 3	Customer A requests 2 units	1 available	DENIED - insufficient resources

8.3 IPC Test Cases

Test	Request	Expected Response	Validates
TC-I1	Alice deposits 500 to #123	Success balance=1500	Message send and process
TC-I2	Dave withdraws 700 from #456 (balance=800)	Success balance=100	Concurrent server access
TC-I3	Server closes queue after all messages	Receive returns false gracefully	Queue close handling

9. Conclusion

This project successfully implements a Concurrent Banking System that integrates five core Operating System concepts in a single C++ application. Each module demonstrates a specific OS mechanism with measurable outcomes.

The scheduling module shows that Priority Scheduling achieves the best average waiting time (4.00 ms) for this workload because high-priority VIP and Premium customers are served first. FCFS provides fairness by arrival order, while Round Robin provides time-sharing but introduces overhead.

The synchronization module confirms that mutex locks and semaphores are essential for correct concurrent banking. Without them, concurrent deposits would cause lost updates and incorrect balances.

The Banker's Algorithm module demonstrates proactive deadlock prevention. By simulating resource allocation before committing, the system avoids unsafe states that would lead to deadlock in a real loan management scenario.

The IPC module shows how a producer-consumer model with a condition-variable-based message queue enables safe communication between customer threads and the bank server without busy waiting.

The memory management module confirms that LRU (8 faults, 33.33% hit ratio) outperforms FIFO (9 faults, 25.00% hit ratio) for the given page reference string, reinforcing the value of locality-aware replacement policies.

Overall, the project demonstrates a practical application of OS theory and provides a foundation for extending the system with real network sockets, persistent storage, or a graphical dashboard.

10. References

1. Abraham Silberschatz, Peter B. Galvin, Greg Gagne - Operating System Concepts, 10th Edition, Wiley.
2. cppreference.com - C++ Standard Library Thread Support: `std::thread`, `std::mutex`, `std::condition_variable`.
3. The Open Group Base Specifications - POSIX Semaphores: `sem_init`, `sem_wait`, `sem_post`.